جامعة مولاي إسماعيل
UNIVERSITÉ MOULAY ISMAÏL

كلية العلوم والتقنيات
FACULTÉ DES SCIENCES ET TECHNIQUES

Année universitaire 2020/2021

POLYCOPIE DE TRAVAUX PRATIQUES CORRIGE

Module : Informatique industrielle

Filière : Master Electronique Et Systèmes Embarqués

Semestre : 9

Pr. Mounir Hanine

FACULTÉ DES SCIENCES ET TECHNIQUES D'ERRACHIDIA - Université Moulay Ismail

Table des matières

Partie I — Initiation à l'environnement de programmation LabVIEW 2
TP1 — Premiers VI et débogage 2
TP2 — Sous-VI 3
TP3 — Boucles et temporisation 4
TP4 — Tableaux et clusters 0
TP5 — Graphes et générateur de signaux 0
TP6 — Structure Condition et nœud de formule 0
TP7 — Chaînes de caractères 0
TP8 — Enregistrement de mesures dans un fichier 0
Partie II — Contrôle d'instruments : générateur SEFRAM 4463 et oscilloscope SEFRAM 5062DC 1
Le banc de mesure 1
Piloter le générateur SEFRAM 4463 pas à pas 1
Niveau 1 — Dialoguer avec l'appareil 1
Niveau 2 — Un bouton, une commande 3
Niveau 3 — Régler le signal 10
Niveau 4 — Vérifier le signal avec l'oscilloscope 15
TP9 — Mise en communication GPIB 16
TP10 — Piloter le générateur et mesurer à l'oscilloscope 17
TP11 — Rapatrier le signal de l'oscilloscope sur le PC 18
Annexe — Aide-mémoire 20
Raccourcis LabVIEW 20
Générateur SEFRAM 4463 (commandes utilisées) 20
Oscilloscope SEFRAM 5062DC 20

Partie I — Initiation à l'environnement de programmation LabVIEW

TP1 — Premiers VI et débogage

Objectif : construire un VI, distinguer commandes et indicateurs, utiliser les outils de débogage.

Travail demandé

1. **Calculatrice.** Créez un VI avec deux commandes numériques A et B et quatre indicateurs : A + B, A − B, A × B et A / B.

    - Q1. Exécutez-le avec A = 12 et B = 4. Notez les quatre résultats.
    - Q2. Exécutez-le avec A = 12 et B = 0. Quel résultat affiche la division ? LabVIEW affiche-t-il un message d'erreur ?

2. **Convertisseur de température.** Créez un VI qui convertit une température en degrés Celsius (commande « T (°C) ») en degrés Fahrenheit (indicateur « T (°F) »). On rappelle : T(°F) = T(°C) × 1,8 + 32.

    - Q3. Complétez le tableau des résultats pour 0 °C, 37 °C et 100 °C.
    - Q4. Enregistrez le VI sous le nom C_to_F.vi (il resservira au TP2).

3. **Débogage.**

    - Q5. Dans la calculatrice, supprimez le fil qui relie B à la division. Que devient la flèche « Exécuter » de la barre d'outils ? Cliquez dessus et recopiez le message affiché. Puis réparez le fil.
    - Q6. Dans le convertisseur, activez l'animation (icône ampoule) et placez une sonde sur le fil qui sort de la multiplication. Pour T = 37 °C, quelle valeur lit la sonde ? Pourquoi n'est-ce pas encore le résultat final ?

Corrigé

- **Calculatrice :** quatre fonctions de la palette Numérique (*Additionner*, *Soustraire*, *Multiplier*, *Diviser*), chacune reliée à A et B. Résultats : 16, 8, 48 et 3. Avec B = 0, la division donne **+Inf** sans message d'erreur : LabVIEW suit la norme des réels IEEE.
- **Convertisseur :** T(°F) = T(°C) × 1,8 + 32. Une fonction *Multiplier* avec une constante 1,8, puis *Additionner* avec une constante 32 (clic droit sur l'entrée › Créer › Constante).

T (°C)	0	37	100
T (°F)	32	98,6	212

Débogage : l'entrée débranchée rend la flèche d'exécution **brisée**. Un clic dessus ouvre la liste des erreurs (« terminal requis non câblé ») ; un double-clic sélectionne la fonction fautive.
Avec l'animation, les valeurs s'affichent le long des fils. Pour T = 37 °C, la sonde placée après la multiplication lit 66,6 : il reste à ajouter 32 pour obtenir le résultat final, 98,6 °F.

TP2 — Sous-VI

Objectif : créer des sous-VI réutilisables (icône, connecteur, documentation) et les appeler depuis un VI principal.

Travail demandé

1. **Transformer C_to_F.vi en sous-VI.** Ouvrez C_to_F.vi (TP1).
   - ❖ Q1. Dessinez une icône portant le texte « °C→°F ».
   - ❖ Q2. Reliez le connecteur : la commande T (°C) sur un terminal de gauche, l'indicateur T (°F) sur un terminal de droite.
   - ❖ Q3. Ecrivez une phrase de documentation qui explique ce que fait le VI. Vérifiez qu'elle apparaît dans l'aide contextuelle (Ctrl+H).

2. **Créer le sous-VI Puissance.vi.** Il reçoit une tension U (en V) et un courant I (en A), et renvoie :
   - ❖ la puissance P = U × I (en W),
   - ❖ la résistance R = U / I (en Ω),
   - ❖ une LED « Circuit ouvert », allumée quand I = 0.

3. **Utiliser le sous-VI.** Créez un VI principal qui appelle Puissance.vi.
   - ❖ Q4. Complétez le tableau ci-dessous avec les valeurs affichées.

U (V)	I (A)	P (W)	R (Ω)	Circuit ouvert
12	0,5	?	?	?
5	0,02	?	?	?
9	0	?	?	?

Corrigé

- ➢ **Icône et connecteur :** clic droit sur l'icône en haut à droite de la face avant › *Modifier l'icône*. Puis *Afficher le connecteur*, modèle 4-2-2-4 : cliquer sur un terminal du connecteur puis sur la commande ou l'indicateur à lui associer. Documentation : Fichier › Propriétés du VI › Documentation. Elle s'affiche dans l'aide contextuelle (Ctrl+H).
- ➢ **Puissance.vi :** *Multiplier* (U × I), *Diviser* (U / I), *Egal à 0 ?* sur I relié à la LED. Connecteur : U et I à gauche ; P, R et la LED à droite.
- ➢ **Appel :** dans le VI principal, Fonctions › Sélectionner un VI..., ou glisser l'icône du sous-VI sur le diagramme.

U (V)	**I (A)**	**P (W)**	**R (Ω)**	**Circuit ouvert**
12	0,5	6	24	éteinte
5	0,02	0,1	250	éteinte
9	0	0	+Inf	allumée

TP3 — Boucles et temporisation

Objectif : utiliser les boucles For et While, les registres à décalage et la temporisation.

Travail demandé

1. **Boucle For : somme et factorielle.** Créez un VI avec une commande entière N et deux indicateurs S et F.
   - ❖ S = 1 + 2 + … + N (somme des N premiers entiers),
   - ❖ F = 1 × 2 × … × N (factorielle de N, notée N!).
   - ❖ Q1. Notez S et F pour N = 5, puis pour N = 10.
   - ❖ Q2. Essayez N = 15 avec F en entier I32. Le résultat est-il juste ? Quel type faut-il choisir pour F ?

2. **Thermomètre simulé avec alarme.** Créez un VI qui, toutes les 250 ms :
   - ❖ tire une température au hasard entre 20 et 30 °C, avec la formule T = 20 + 10 × (nombre aléatoire entre 0 et 1) ;
   - ❖ affiche T sur un graphe déroulant ;
   - ❖ allume une LED « Alarme » si T dépasse un seuil réglable (par exemple 28 °C) ;
   - ❖ compte le nombre d'alarmes depuis le démarrage ;
   - ❖ un bouton STOP arrête le programme.
   - ❖ Q3. Laissez tourner une minute avec un seuil de 28 °C. Combien d'alarmes comptez-vous ?
3. **Moyenne glissante.**
   - ❖ Q4. Ajoutez au graphe une seconde courbe : la moyenne des 3 dernières températures. Laquelle des deux courbes varie le moins ? Pourquoi ?

4. **Rôle de l'attente.**
   - ❖ Q5. Supprimez l'attente de 250 ms et relancez le VI. Ouvrez le Gestionnaire des tâches de Windows : que devient l'occupation du processeur ? Remettez l'attente.

Corrigé

- **Boucle For :** N câblé sur le terminal N. Deux registres à décalage (clic droit sur le bord › Ajouter un registre à décalage) : S initialisé à 0, F initialisé à 1. Le compteur i va de 0 à N − 1 : on utilise k = i + 1. A chaque tour, S ← S + k et F ← F × k.

N	S	N!
5	15	120
10	55	3 628 800

F doit être de type DBL ou U64 : 13! dépasse déjà la capacité d'un entier I32.

- **Thermomètre :** une boucle While contient *Nombre aléatoire (0-1)* × 10 + 20 → T, relié au graphe déroulant. *Supérieur ?* compare T au seuil et allume la LED. Le compteur est un registre à décalage initialisé à 0 : *Sélectionner* ajoute 1 si l'alarme est vraie, 0 sinon. *Attendre un multiple de ms* = 250. Le bouton STOP est câblé au terminal conditionnel, **à l'intérieur** de la boucle.

- **Moyenne glissante :** étirer le registre à décalage de gauche vers le bas pour obtenir T(i−1) et T(i−2). Moyenne = (T + T(i−1) + T(i−2)) / 3. *Assembler* T et la moyenne en un cluster : le graphe déroulant trace alors deux courbes. La moyenne fluctue nettement moins que T.

- **Sans attente :** la boucle tourne aussi vite que possible et un cœur du processeur monte à près de 100 %. Toute boucle While d'interface doit contenir une attente.

TP4 — Tableaux et clusters

Objectif : créer des tableaux par auto-indexation, utiliser les fonctions de tableaux, regrouper des données dans un cluster.

Travail demandé

1. **Statistiques sur des mesures.** Placez sur le diagramme une constante tableau contenant 5 tensions : 12,5 ; 13,1 ; 12,8 ; 14,0 ; 13,6 (en V).
   - ❖ Q1. Affichez la plus grande valeur et sa position (indice) dans le tableau.
   - ❖ Q2. Affichez la plus petite valeur, la moyenne et le nombre d'éléments.
   - ❖ Q3. Affichez le tableau trié par ordre croissant.

2. **Auto-indexation.** Une boucle For de 10 tours tire à chaque tour un nombre aléatoire.
   - ❖ Q4. Reliez ce nombre à un indicateur placé hors de la boucle. Qu'obtient-on : un nombre ou un tableau ? Combien d'éléments ?
   - ❖ Q5. Clic droit sur le tunnel de sortie › désactivez l'indexation. Qu'obtient-on maintenant ?
3. **Tableau à deux dimensions.**
   - ❖ Q6. Avec deux boucles For imbriquées (5 tours chacune), construisez la table de multiplication de 1 à 5. Quelle valeur contient la dernière case ?
4. **Photodiode à 4 quadrants.** Une photodiode est découpée en 4 zones A, B, C, D qui fournissent chacune un courant (en µA) selon la position du faisceau laser. On calcule la position (X, Y) du spot avec Σ = A + B + C + D :

   $$X = ((A + D) - (B + C)) / \Sigma$$
   $$Y = ((A + B) - (C + D)) / \Sigma$$

   - ❖ Q7. Créez quatre commandes A, B, C, D et calculez X et Y. Pour A = 40, B = 20, C = 10, D = 30, quelles valeurs trouvez-vous ?
   - ❖ Q8. Regroupez X et Y dans un cluster et affichez la position du spot sur un graphe XY.

Corrigé

- **Statistiques :** *Max & Min d'un tableau*, *Moyenne* (palette Probabilités et statistiques), *Taille d'un tableau*, *Trier un tableau 1D*. Aucune boucle n'est nécessaire.

Résultat	**Valeur**
Maximum (indice)	14,0 V (indice 3)
Minimum (indice)	12,5 V (indice 0)
Moyenne	66,0 / 5 = 13,2 V
Taille	5
Tableau trié	[12,5 ; 12,8 ; 13,1 ; 13,6 ; 14,0]

- **Auto-indexation :** activée, le tunnel de sortie (crochets [ ]) donne un tableau de 10 valeurs. Désactivée (clic droit sur le tunnel), seule la dernière valeur sort. Sur une boucle For, l'auto-indexation est activée par défaut ; sur une boucle While, elle est désactivée.

- **Tableau 2D :** boucle externe i (lignes), boucle interne j (colonnes), valeur (i + 1) × (j + 1). Les deux tunnels de sortie auto-indexés produisent un tableau 5 × 5 dont la dernière case vaut 25.

- **Photodiode :** avec Σ = A + B + C + D :
  - ❖ X = ((A + D) − (B + C)) / Σ
  - ❖ Y = ((A + B) − (C + D)) / Σ

Exemple : A = 40, B = 20, C = 10, D = 30 donnent Σ = 100, X = (70 − 30) / 100 = **0,4** et Y = (60 − 40) / 100 = **0,2**. Le graphe XY attend un cluster de deux tableaux : transformer X et Y en tableaux d'un élément avec *Construire un tableau*, puis les *Assembler* (X sur l'entrée du haut). Pour suivre le spot en continu, mettre le tout dans une boucle While.

TP5 — Graphes et générateur de signaux

Objectif : choisir le bon indicateur graphique (graphe, graphe déroulant, graphe XY) et construire un générateur de signaux logiciel.

Travail demandé

1. **Générateur de signaux logiciel.** Créez un VI avec trois commandes : nombre d'échantillons N (valeur 1 000), nombre de périodes k (valeur 3) et amplitude A (valeur 2). Ajoutez un menu déroulant « Signal » avec trois choix : Sinus, Carré, Les deux. Le signal calculé s'affiche sur un graphe.

    - Q1. Choisissez Sinus. Combien de périodes voyez-vous ? Entre quelles valeurs le signal varie-t-il ?
    - Q2. Choisissez Les deux. Combien de courbes le graphe affiche-t-il ?

2. **Ajouter du bruit.**
    - Q3. Ajoutez une case à cocher « Bruit ». Quand elle est cochée, un bruit gaussien d'écart-type 0,1 s'ajoute au signal. Décrivez la différence sur le graphe.

3. **Graphe déroulant.**

    - Q4. Dans une boucle While cadencée à 100 ms, tracez en direct deux courbes : sin(i/10) et cos(i/10), où i est le compteur de la boucle. En quoi l'affichage diffère-t-il de celui d'un graphe simple ?

Corrigé

- **Calcul du sinus :** une boucle For de N tours ; pour l'indice i, $y = A \times \sin(2\pi \times k \times i / N)$. L'auto-indexation produit le tableau des N valeurs.

- **Calcul du carré :** même boucle, $y = A$ si $\sin(2\pi \times k \times i / N) \geq 0$, sinon $-A$ (fonction *Sélectionner*). On peut aussi utiliser *Sine Pattern.vi* et *Square Wave.vi* de la palette Traitement du signal.

- **Choix du signal :** une structure Condition reliée au menu déroulant. Cas « Sinus » et « Carré » : un tableau 1D vers le graphe. Cas « Les deux » : *Construire un tableau* avec les deux tableaux donne un tableau 2D, donc deux courbes.

- **Bruit :** *Gaussian White Noise.vi* (N points, écart-type 0,1) ajouté au signal par *Additionner*, qui est polymorphe (tableau + tableau). Une structure Condition sur la case « Bruit » choisit le signal bruité ou non.

- 3 périodes complètes sur 1 000 points, entre −2 et +2.

- **Graphe déroulant :** *Assembler* sin(i/10) et cos(i/10) en un cluster à chaque tour : le graphe déroulant ajoute un point par courbe et garde l'historique (1 024 points par défaut).

Indicateur	Reçoit	Usage
Graphe	un tableau complet	signal calculé ou acquis en bloc
Graphe déroulant	un point par tour de boucle	surveillance en direct
Graphe XY	deux tableaux X et Y	courbes paramétriques, caractéristiques

TP6 — Structure Condition et nœud de formule

Objectif : utiliser la structure Condition et le nœud de formule.

Travail demandé

1. **Sélecteur de multimètre (structure Condition).** Placez un bouton rotatif réglable de 0 à 6 et un indicateur texte « Mode ». Le texte affiché dépend de la position :

Position	Texte affiché
0	Inconnu
1	Voltmètre DC
2	Voltmètre AC
3	Ampèremètre DC
4	Ampèremètre AC
5	Ohmmètre
6	Wattmètre

   - ❖ Q1. Que se passe-t-il si on tape 9 dans le bouton ? Comment prévoir ce cas ?

2. **Nœud de formule.** Avec quatre commandes a, b, c et x, calculez :
   - $y = a \times x^2 + b \times x + c$;
   - $z = \sqrt{y}$ si y est positif ou nul, et z = 0 sinon.

   - ❖ Q2. Avec a = 1, b = −3 et c = 2, complétez y et z pour x = 0, x = 1,5 et x = 4.

Corrigé

- **Sélecteur :** le bouton rotatif (représentation I32) est câblé au terminal de sélection d'une structure Condition. On ajoute des cas jusqu'à 6 (clic droit › Ajouter un cas après) ; chacun contient une constante chaîne reliée à l'indicateur « Mode ». Le cas 0 est défini « Par défaut » pour couvrir toute autre valeur.

- **Nœud de formule :** entrées a, b, c, x et sorties y, z créées par clic droit sur le bord du nœud. Code :

```
y = a*x**2 + b*x + c;
if (y >= 0) z = sqrt(y);
else z = 0;
```

x	y	z
0	2	1,414

x	y	z
1,5	−0,25	0
4	6	2,449

TP7 — Chaînes de caractères

Objectif : construire et décoder les chaînes échangées avec les instruments.

Travail demandé

1. **Fabriquer une commande pour un instrument.** A partir d'une fréquence f (en Hz) et d'une amplitude A (en V), le VI doit produire une chaîne de caractères de la forme `:FREQ 1000.000;:VOLT:AMPL 2.50`.
   - ❖ Q1. Quelle chaîne obtenez-vous pour f = 1 000 et A = 2,5 ?
   - ❖ Q2. Les instruments exigent un point décimal. Sur un PC réglé en français, qu'écrit LabVIEW par défaut ? Comment forcer le point ?

2. **Lire la réponse d'un instrument.** Un multimètre renvoie la chaîne `+1.20664E+1VDC`.
   - ❖ Q3. Séparez la valeur numérique de l'unité. Quelle tension, en volts, cette chaîne représente-t-elle ?
   - ❖ Q4. Affichez cette tension avec 3 décimales.

3. **Coder un message.** On code un texte en ajoutant 3 au code ASCII de chaque lettre.
   - ❖ Q5. Codez le mot « LabVIEW ». Quel mot codé obtenez-vous ?
   - ❖ Q6. Ecrivez la partie qui décode et vérifiez qu'on retrouve « LabVIEW ».

4. **Horodater une mesure.**
   - ❖ Q7. Construisez une ligne de texte composée de la date, de l'heure, d'une tabulation et d'une valeur avec 3 décimales, par exemple : `15/03/2023 14:05:12 <tab> 12.066`. Cette ligne servira au TP8.

Corrigé

- **Commande :** *Formater en chaîne* avec la chaîne de format `%.;:FREQ %.3f;:VOLT:AMPL %.2f`. Le préfixe `%.;` impose le point décimal.
- **Décodage :** *Balayer une chaîne* avec le format `%.;%f%s`. Sortie 1 = 12,0664 (DBL), sortie 2 = « VDC ». Puis *Formater en chaîne* avec `%.3f V` donne « 12.066 V ».
- **Codage :** *Chaîne en tableau d'octets* → *Additionner* 3 (polymorphe : tout le tableau d'un coup) → *Tableau d'octets en chaîne*. Pour décoder, soustraire 3.

Message	Codes ASCII	Codes + 3	Message codé
LabVIEW	76 97 98 86 73 69 87	79 100 101 89 76 72 90	OdeYLHZ

Horodatage : *Formater date/heure en chaîne* avec `%d/%m/%Y %H:%M:%S`, puis *Concaténer des chaînes* avec la constante *Tabulation* et la valeur formatée par `%.;%.3f`. Cette ligne servira au TP8.

TP8 — Enregistrement de mesures dans un fichier

Objectif : enregistrer des mesures horodatées dans un fichier tableur lisible par Excel, puis le relire.

Travail demandé

1. **Simuler deux mesures.** Placez une jauge « T » (température, de 0 à 100 °C) et un bouton rotatif « P » (pression, de 0 à 10 bar). Toutes les 500 ms, le VI lit T et P et les trace sur un graphe déroulant à deux courbes.
2. **Enregistrer dans un fichier.** Le fichier commence par deux lignes d'en-tête : la date du jour, puis les noms des colonnes « tps », « T » et « P ». Ensuite, à chaque mesure, le VI ajoute une ligne : temps écoulé en secondes, T, P, séparés par des tabulations.
   - ❖ Q1. Faites tourner le VI 10 secondes en bougeant la jauge et le bouton. Combien de lignes de mesures le fichier contient-il ?
   - ❖ Q2. Pourquoi vaut-il mieux ouvrir le fichier une seule fois avant la boucle plutôt qu'à chaque mesure ?

3. **Ouvrir le fichier dans Excel.**
   - ❖ Q3. Ouvrez le fichier dans Excel. Les nombres sont-ils bien reconnus comme des nombres ? Si non, pourquoi ?

4. **Relire le fichier.**
   - ❖ Q4. Ecrivez un second VI qui relit le fichier et trace T et P en fonction du temps sur un graphe XY.

Corrigé

- **Structure efficace (E/S de bas niveau) :** avant la boucle, *Ouvrir/Créer/Remplacer un fichier* (opération « remplacer ou créer ») puis *Ecrire dans un fichier texte* de l'en-tête. Dans la boucle, *Ecrire dans un fichier texte* de la ligne de mesures. Après la boucle, *Fermer un fichier*. Le fichier n'est ouvert qu'une fois : c'est bien plus rapide que de le rouvrir à chaque mesure.
- **Ligne de mesures :** *Formater en chaîne* avec `%.;%.1f\t%.2f\t%.2f\n`, où `\t` est la tabulation et `\n` le retour à la ligne. Temps écoulé = compteur i × 0,5 s.
- **En-tête :** date du jour (*Formater date/heure en chaîne*) sur la première ligne, puis `tps\tT\tP\n`.
- **Contenu attendu du fichier :**

```
15/03/2023
tps   T     P
0.0   23.40 1.20
0.5   23.50 1.20
1.0   23.70 1.30
```

- **Lecture :** *Lire un fichier tableur* (format `%.;%f`, délimiteur tabulation, lignes à ignorer : 2) renvoie un tableau 2D. *Indexer un tableau* extrait les colonnes 1 (T) et 2 (P), et la colonne 0 (temps) sert d'abscisse sur un graphe XY.

Partie II — Contrôle d'instruments : générateur SEFRAM 4463 et oscilloscope SEFRAM 5062DC

Le banc de mesure

Le PC commande les deux appareils par un seul bus GPIB : il règle le signal sur le générateur, puis le mesure ou le récupère sur l'oscilloscope.

Appareil	Rôle	Adresse GPIB	Réglage en face avant	Ressource VISA
SEFRAM 4463	Générateur de fonctions arbitraires	9 (usine)	UTILITY › F1 GPIB	`GPIB0::9::INSTR`
SEFRAM 5062DC	Oscilloscope 2 voies, 60 MHz	7 (à régler)	UTILITY › F2, F1 : GPIB, F2 : adresse	`GPIB0::7::INSTR`

Câblage

- **GPIB :** PC → générateur → oscilloscope (les câbles se chaînent). Deux appareils ne doivent jamais avoir la même adresse.
- **Signal :** sortie OUTPUT du générateur → té BNC → entrée CH1 de l'oscilloscope, avec un bouchon 50 Ω sur le té. Sans bouchon, l'oscilloscope (1 MΩ) lit environ le **double** de l'amplitude programmée, car le générateur affiche sa tension pour une charge de 50 Ω.

Règles communes (voir les manuels de programmation)

- Chaque commande se termine par un saut de ligne LF (`\n`).
- Les nombres s'écrivent avec un point décimal : format `%.;` dans *Formater en chaîne* (TP7).
- Après `*RST`, la sortie du générateur est coupée : toujours renvoyer `:OUTP ON`.
- Amplitude et offset du générateur : Vcc/2 + |offset| ≤ 5 V sur 50 Ω.

Piloter le générateur SEFRAM 4463 pas à pas

Niveau 1 — Dialoguer avec l'appareil

Les exercices 1 à 6 se font dans NI MAX (« Communiquer avec l'instrument », ressource `GPIB0::9::INSTR`) : on tape une commande, on lit la réponse.

Exercice 1 — Identification. Dans NI MAX, ouvrez « Communiquer avec l'instrument » sur le générateur (GPIB0::9::INSTR).

- Q1. Quelle commande demande à un appareil de donner son nom ? Envoyez-la et recopiez la réponse.
- Q2. La réponse contient quatre informations séparées par des virgules. Nommez chacune d'elles.
- Q3. Quelle fonction LabVIEW permet de découper cette réponse en un tableau de quatre chaînes ?

Corrigé : `*IDN?`. Réponse : `SEFRAM,SEFRAM 4463,0,V1.1`. Les quatre champs, séparés par des virgules, sont le fabricant (SEFRAM), le modèle (SEFRAM 4463), le numéro de série (0) et la version du micrologiciel (V1.1). Dans LabVIEW, *Chaîne en tableau tableur* avec le délimiteur « , » donne un tableau de 4 chaînes. Les commandes qui commencent par `*` sont communes à tous les instruments IEEE 488.2.

Exercice 2 — Autotest. Envoyez au générateur la commande `*TST?` (autotest).

- Q1. Quelle réponse obtenez-vous ? Que signifie-t-elle ?
- Q2. Pendant l'autotest, l'appareil ne traite aucune autre commande. Que doit faire le programme avant d'envoyer la commande suivante ?

Corrigé : `*TST?`. La réponse `0` signifie que l'autotest est réussi ; toute autre valeur signale un défaut. Pendant le test, l'appareil ne traite pas les commandes suivantes : il faut lire la réponse (et prévoir un délai d'attente VISA suffisant) avant de continuer.

Exercice 3 — Remise à zéro. Envoyez `*RST`, puis les quatre requêtes `:FUNC?`, `:FREQ?`, `:VOLT:AMPL?` et `:OUTP?`.

- Q1. Recopiez les quatre réponses dans un tableau (requête / réponse).
- Q2. Dans quel état est la sortie du générateur après `*RST` ? Quelle commande faut-il envoyer ensuite pour retrouver un signal ?

Corrigé :

Requête	Réponse après *RST
`:FUNC?`	SIN
`:FREQ?`	1 MHz (écrit en notation scientifique)
`:VOLT:AMPL?`	5 V crête à crête
`:OUTP?`	0, sortie coupée

Remarque : `*RST` coupe la sortie. Il faut toujours renvoyer `:OUTP ON` ensuite.

Exercice 4 — Lire les erreurs. La requête `:SYST:ERR?` lit la file d'erreurs de l'appareil.

- Q1. Envoyez `:FREQUENCE 1000` (commande volontairement mal écrite), puis `:SYST:ERR?`. Recopiez le code et le message reçus.
- Q2. Envoyez `:FREQ 50MHZ`, puis `:SYST:ERR?`. Recopiez le code et le message. Pourquoi cette commande est-elle refusée ? (Fréquence maximale du générateur : 31 MHz.)
- Q3. Que répond `:SYST:ERR?` quand il n'y a plus d'erreur ? Donnez deux façons de vider la file.

Corrigé : la requête est `:SYST:ERR?`. Chaque commande refusée laisse un code négatif suivi d'un libellé. La norme SCPI prévoit −13x pour une commande inconnue (par exemple −113 « Undefined header ») et −22x pour une valeur hors limites, ici 50 MHz > 31 MHz. Relevez les libellés exacts renvoyés par l'appareil. La file se vide en lisant `:SYST:ERR?` jusqu'à obtenir `0,"No error"`, ou d'un coup avec `*CLS`.

Exercice 5 — Premier signal. On veut obtenir un sinus de 1 kHz, 2 Vcc, offset 0 V, sortie active.

- Q1. Ecrivez les cinq commandes nécessaires, une par ligne : forme, fréquence, amplitude, offset, sortie.
- Q2. Réécrivez-les en une seule ligne, séparées par « ; ». Envoyez cette ligne et vérifiez le signal sur l'oscilloscope.
- Q3. `:FREQ 1KHZ` et `:FREQ 1000` donnent-ils le même résultat ? Pourquoi ?

Corrigé :

```
:FUNC SIN
:FREQ 1KHZ
:VOLT:AMPL 2
:VOLT:OFFS 0
:OUTP ON
```

En une ligne : `:FUNC SIN;:FREQ 1KHZ;:VOLT:AMPL 2;:VOLT:OFFS 0;:OUTP ON`. Les unités sont acceptées en suffixe (HZ, KHZ, MHZ). Sans unité, la valeur est en hertz : `:FREQ 1000` a le même effet.

Exercice 6 — Relire les réglages. On veut vérifier que les réglages de l'exercice 5 ont bien été appliqués.

- Q1. Ecrivez la requête qui relit chaque réglage (forme, fréquence, amplitude, offset, sortie). Envoyez-les et recopiez les réponses.
- Q2. La réponse à `:FREQ?` est une chaîne de caractères. Quelle fonction LabVIEW la convertit en nombre, pour la comparer à la consigne ?

Corrigé : ajouter `?` à chaque commande : `:FUNC?` → SIN, `:FREQ?` → 1000 en notation scientifique, `:VOLT:AMPL?` → 2.00, `:VOLT:OFFS?` → 0.00, `:OUTP?` → 1. Dans LabVIEW, *Chaîne fractionnaire en nombre* (ou *Balayer une chaîne* avec `%.;%f`) convertit la réponse en nombre DBL ; comparer ensuite à la consigne.

Niveau 2 — Un bouton, une commande

Exercice 7 — Mon premier VI de dialogue (sans bouton)

Ce qu'on apprend : les quatre fonctions VISA de base et le fil d'erreur.

Enoncé. Créez un nouveau VI nommé Ex07_Identifier.vi.

1. Sur la face avant, placez une commande « nom de ressource VISA » nommée Générateur (valeur `GPIB0::9::INSTR`) et un indicateur chaîne nommé Identité.
2. Dans le diagramme, utilisez les fonctions VISA pour : ouvrir la liaison, envoyer `*IDN?`, lire la réponse, fermer la liaison. La réponse s'affiche dans Identité.
3. Lancez le VI une fois (Ctrl+R).
    - ❖ Q1. Quelle réponse s'affiche dans Identité ?
    - ❖ Q2. A quoi servent le fil violet et le fil jaune qui relient les fonctions VISA entre elles ?

Corrigé.

Face avant :

Objet	Nom	Réglage
Commande nom de ressource VISA	Générateur	`GPIB0::9::INSTR` (clic droit › Opérations sur les données › Valeur actuelle par défaut)
Indicateur chaîne	Identité	—

Diagramme, de gauche à droite :

1. *VISA Open* : entrée « nom de ressource VISA » reliée à la commande Générateur.
2. *VISA Write* : entrée « tampon d'écriture » = constante chaîne `*IDN?\n`. Pour taper `\n`, clic droit sur la constante › *Affichage des codes* « \ ».
3. *VISA Read* : entrée « nombre d'octets » = 100. La sortie « tampon de lecture » va vers l'indicateur Identité.
4. *VISA Close*.
5. *Gestionnaire d'erreur simple* (palette Dialogue et interface utilisateur).

La référence VISA (fil violet) et le fil d'erreur (fil jaune) passent d'une fonction à la suivante : c'est ce chaînage qui impose l'ordre d'exécution.

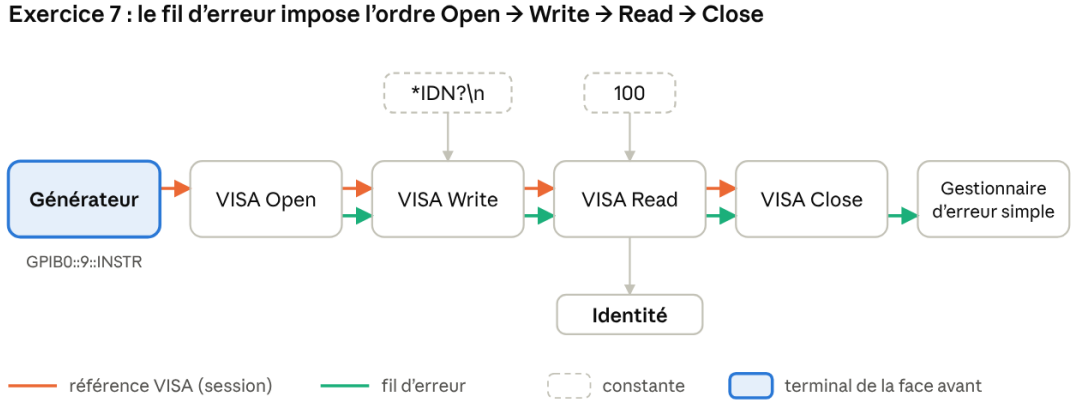


Diagramme de l'exercice 7 · chaîne VISA linéaire

Schéma du diagramme : les blocs se lisent de gauche à droite ; la réponse de *VISA Read* descend vers l'indicateur Identité.

Identité = `SEFRAM,SEFRAM 4463,0,V1.1`.

Erreurs fréquentes :

- **Erreur −1073807339 (timeout) :** la commande est partie sans `\n` (la constante n'est pas en affichage codes « \ »), ou l'adresse GPIB est fausse.
- **Réponse tronquée :** le nombre d'octets demandé est trop petit. 100 suffit ici ; la lecture s'arrête d'elle-même au saut de ligne final.
- **Erreur « ressource introuvable » :** le générateur n'apparaît pas dans NI MAX. Reprendre le TP9.

Exercice 8 — Un bouton « Identifier »

Ce qu'on apprend : faire tourner le VI en continu et n'envoyer la commande qu'au clic.

Enoncé. Enregistrez une copie de l'exercice 7 sous le nom Ex08_Bouton.vi.

1. Ajoutez un bouton « Identifier » et un bouton STOP.
2. Modifiez le diagramme pour que le VI tourne en permanence, jusqu'au clic sur STOP.
3. La commande `*IDN?` ne doit partir qu'au clic sur Identifier, une seule fois par clic.
    - ❖ Q1. Quelle action mécanique faut-il choisir pour le bouton Identifier ? Que se passe-t-il avec l'action « Switch » ?
    - ❖ Q2. Pourquoi ajoute-t-on une attente de 100 ms dans la boucle ?

Corrigé.

Face avant : ajouter un bouton-poussoir « Identifier » (palette Booléen) et un bouton STOP.

Régler l'action mécanique du bouton Identifier : clic droit › *Action mécanique* › **Latch When Pressed** (verrouillage à l'appui). Avec ce réglage, le bouton reste « Vrai » jusqu'à ce que le programme le lise, puis revient tout seul à « Faux ». Un clic correspond donc à un seul envoi.

Diagramme :

1. Entourer tout le code de l'exercice 7 d'une **boucle While** (palette Structures).
2. Dans la boucle, ajouter une **structure Condition**. Relier le terminal du bouton Identifier à son sélecteur « ? ».
3. Déplacer *VISA Open*, *VISA Write*, *VISA Read* et *VISA Close* dans le cas **Vrai**. Dans le cas **Faux**, ne rien mettre : les fils d'erreur traversent simplement la structure.
4. Relier le bouton STOP au terminal conditionnel de la boucle (l'icône rouge en bas à droite).
5. Ajouter *Attendre (ms)* = 100 dans la boucle.
6. Placer le *Gestionnaire d'erreur simple* après la boucle.

L'indicateur Identité reste vide au démarrage, puis se remplit au premier clic. Chaque nouveau clic renvoie la commande ; on le voit en effaçant l'indicateur entre deux clics.

Pourquoi l'attente de 100 ms ? Sans elle, la boucle tourne des milliers de fois par seconde pour rien et occupe tout un cœur du processeur. Un humain ne clique pas plus de 10 fois par seconde : 100 ms suffisent.

Exercice 9 — Un second bouton « Autotest » dans le même programme

Ce qu'on apprend : ajouter une deuxième action sans casser la première.

Enoncé. Enregistrez une copie de l'exercice 8 sous le nom Ex09_Deux_boutons.vi.

1. Ajoutez un bouton « Autotest », un indicateur chaîne « Résultat autotest » et une LED verte « Appareil OK ».
2. Au clic sur Autotest, le VI envoie `*TST?` et lit la réponse.
3. Si la réponse vaut 0, la LED s'allume et l'indicateur affiche « Réussi ». Sinon, la LED reste éteinte et l'indicateur affiche « Echec ».
4. Le bouton Identifier doit toujours fonctionner.
    - ❖ Q1. Combien de temps faut-il pour obtenir le résultat de l'autotest ?
    - ❖ Q2. Pourquoi relie-t-on le fil d'erreur de la première structure Condition à la seconde ?

Corrigé.

Face avant : un bouton « Autotest » (Latch When Pressed), un indicateur chaîne « Résultat autotest » et une LED « Appareil OK ».

Diagramme :

1. Dans la même boucle While, à côté de la première structure Condition, en ajouter une seconde, reliée au bouton Autotest.
2. Cas **Vrai** : *VISA Open* → *VISA Write* `*TST?\n` → *VISA Read* (10 octets) → *VISA Close*.
3. Convertir la réponse : *Chaîne décimale en nombre* (palette Chaîne › Conversion nombre/chaîne).
4. *Egal à 0 ?* sur ce nombre → LED « Appareil OK ».
5. *Sélectionner* (palette Comparaison) : si la LED est vraie, « Réussi » ; sinon « Echec » → indicateur Résultat autotest.
6. Relier le fil d'erreur de la première structure Condition à l'entrée de la seconde : les deux actions ne s'exécutent jamais en même temps sur le bus GPIB.

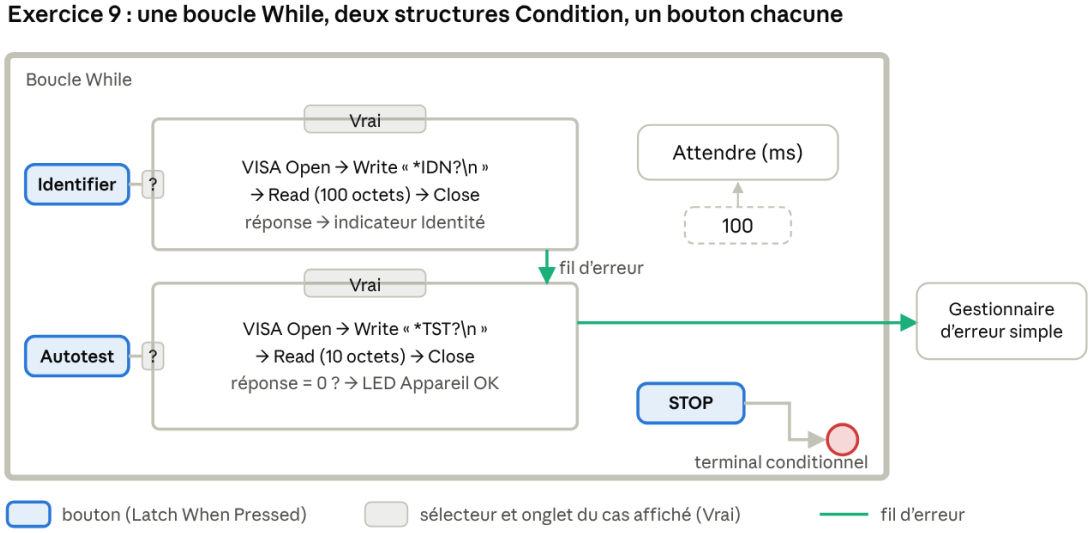


Diagramme de l'exercice 9 · boucle While et deux structures Condition

Schéma du diagramme (cas Vrai affiché ; les cas Faux sont vides, seul le fil d'erreur les traverse). Sans la seconde structure, c'est exactement le diagramme de l'exercice 8.

un clic sur Autotest affiche « Réussi » et allume la LED verte en une à deux secondes, le temps du test interne. Le bouton Identifier fonctionne toujours.

Remarque : on voit déjà un défaut de cette méthode. Chaque bouton répète Open, Write, Read et Close, et la boucle relit tous les boutons 10 fois par seconde. Les exercices 11 et 12 corrigent ces deux points.

Exercice 10 — Un troisième bouton « Reset » et l'état de la sortie

Ce qu'on apprend : enchaîner deux commandes dans une même action, une commande sans réponse suivie d'une requête.

Enoncé. Enregistrez une copie de l'exercice 9 sous le nom Ex10_Trois_boutons.vi.

1. Ajoutez un bouton « Reset » et une LED « Sortie active ».
2. Au clic sur Reset, le VI envoie `*RST`, puis la requête `:OUTP?`. La LED s'allume si la réponse vaut 1.
    - ❖ Q1. Après un clic sur Reset, la LED est-elle allumée ou éteinte ? Pourquoi ?
    - ❖ Q2. Faut-il faire un *VISA Read* juste après `*RST` ? Que se passe-t-il si on le fait ?

Corrigé.

Face avant : un bouton « Reset » (Latch When Pressed) et une LED « Sortie active ».

Diagramme, dans une troisième structure Condition reliée au bouton Reset, cas Vrai :

1. *VISA Open*.
2. *VISA Write* `*RST\n`. C'est une commande : l'appareil ne répond rien, on ne fait donc **pas** de *VISA Read* juste après.
3. *VISA Write* `:OUTP?\n`, puis *VISA Read* (10 octets).
4. *Chaîne décimale en nombre*, puis *Egal à 1 ?* → LED Sortie active.
5. *VISA Close*.

Après Reset, la LED est **éteinte** : `*RST` coupe la sortie du générateur. Sur la face avant de l'appareil, le voyant de sortie s'éteint aussi.

Erreur fréquente : faire un *VISA Read* après `*RST`. Comme l'appareil ne répond rien, la lecture attend jusqu'au timeout (10 s par défaut) puis renvoie une erreur. **Règle à retenir : on ne lit qu'après une commande qui se termine par « ? ».**

Exercice 11 — Remplacer la scrutation par une structure Evénement

Ce qu'on apprend : réagir aux clics au lieu de relire les boutons en permanence.

Enoncé. Enregistrez une copie de l'exercice 10 sous le nom Ex11_Evenements.vi.

1. Supprimez les trois structures Condition et l'attente de 100 ms.
2. Remplacez-les par une seule structure Evénement, avec un cas pour chaque bouton : Identifier, Autotest, Reset et STOP.
3. Vérifiez que le programme se comporte exactement comme à l'exercice 10.
    - ❖ Q1. Où faut-il placer le terminal de chaque bouton ? Que se passe-t-il s'il reste hors de son cas ?
    - ❖ Q2. Quel avantage a cette version par rapport à celle de l'exercice 10 ?

Corrigé.

1. Dans la boucle While, supprimer *Attendre (ms)* et placer une **structure Evénement** (palette Structures).
2. Clic droit sur son bord › *Ajouter un cas d'événement* : source « Identifier », événement « Valeur changée ». Y déplacer le code du cas Vrai de l'exercice 8.
3. Ajouter de même un cas « Autotest : Valeur changée » et un cas « Reset : Valeur changée ».
4. Ajouter un cas « STOP : Valeur changée » : une constante Vrai y est reliée au terminal conditionnel de la boucle. Dans les autres cas, ce tunnel reçoit Faux (clic droit sur le tunnel › *Utiliser la valeur par défaut si non câblé*).
5. **Placer le terminal de chaque bouton à l'intérieur de son cas.** Un bouton Latch ne revient à Faux que lorsque son terminal est lu.
6. Supprimer les trois anciennes structures Condition.

Cas de la structure Evénement	Action
Identifier : Valeur changée	`*IDN?` → indicateur Identité
Autotest : Valeur changée	`*TST?` → Résultat autotest, LED Appareil OK
Reset : Valeur changée	`*RST` puis `:OUTP?` → LED Sortie active
STOP : Valeur changée	arrêt de la boucle

Exercice 11 : une structure Événement, un cas par bouton

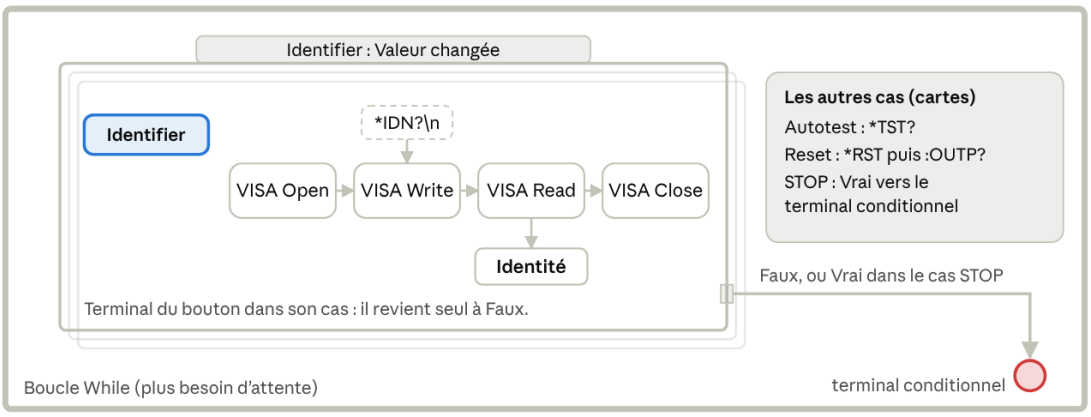


Diagramme de l'exercice 11 · structure Evénement dans la boucle While

Schéma du diagramme : la structure Evénement est un jeu de cartes, un seul cas visible à la fois. Les flèches entre les blocs VISA portent à la fois la référence VISA et le fil d'erreur.

Le programme se comporte comme à l'exercice 10, mais il n'occupe plus le processeur en attendant un clic. Le diagramme est plus lisible : une action = un cas.

Erreur fréquente : laisser le terminal d'un bouton Latch hors de son cas d'événement. Le bouton reste alors enfoncé et l'événement ne se reproduit plus.

Exercice 12 — Une seule session et un sous-VI « Interroger »

Ce qu'on apprend : ouvrir la liaison une fois pour toutes et factoriser le code répété dans un sous-VI.

Enoncé. Enregistrez une copie de l'exercice 11 sous le nom Ex12_Session.vi.

1. Ouvrez la session VISA une seule fois, avant la boucle, et fermez-la une seule fois, après la boucle.
2. Créez un sous-VI Interroger.vi. Entrées : référence VISA, Commande (chaîne), error in. Sorties : référence VISA, Réponse (chaîne), error out. Il envoie la commande et ne lit une réponse que si la commande contient « ? ».
3. Réécrivez les cas Identifier, Autotest et Reset en appelant Interroger.vi.

- ❖ Q1. Comment la référence VISA passe-t-elle d'un tour de boucle au suivant ?
- ❖ Q2. Comparez la longueur du diagramme et la vitesse des réponses avec l'exercice 11.

Corrigé.

Interroger.vi :

Connecteur	Terminal
Entrées (gauche)	référence VISA, Commande (chaîne), error in
Sorties (droite)	référence VISA (sortie), Réponse (chaîne), error out

Diagramme du sous-VI : *Concaténer des chaînes* (Commande + constante *Saut de ligne*) → *VISA Write*. Ensuite, *Rechercher une sous-chaîne* (« ? » dans Commande) : si l'offset trouvé est ≥ 0, une structure Condition fait un *VISA Read* de 1 000 octets ; sinon, elle renvoie une chaîne vide. Pas de *VISA Open* ni de *VISA Close* dans le sous-VI.

Diagramme principal :

1. **Avant la boucle :** *VISA Open* sur Générateur.
2. La référence VISA et le fil d'erreur entrent dans la boucle par des **registres à décalage**, pour être transmis d'un tour au suivant.
3. Chaque cas d'événement appelle Interroger.vi une ou deux fois. Par exemple, cas Reset : Interroger(`*RST`) puis Interroger(`:OUTP?`).
4. **Après la boucle :** *VISA Close*, puis *Gestionnaire d'erreur simple*.

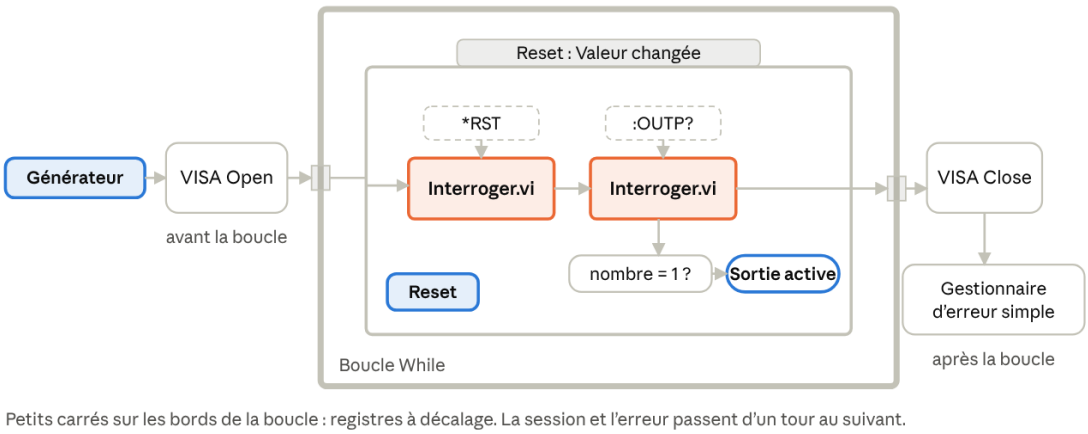


Diagramme de l'exercice 12 · session unique et sous-VI Interroger

Schéma du diagramme principal, cas Reset affiché. Les cas Identifier et Autotest ont la même forme, avec un seul appel à Interroger.vi.

Même comportement qu'à l'exercice 11, avec un diagramme deux fois plus court. Les réponses arrivent un peu plus vite, car la session n'est plus rouverte à chaque clic.

En résumé : un seul programme qui tourne en continu, une session ouverte une fois, un cas d'événement par bouton et un sous-VI de dialogue. **Tous les exercices suivants partent de ce modèle** : chaque nouveau bouton = un nouveau cas d'événement qui appelle Interroger.vi.

Niveau 3 — Régler le signal

On part d'Ex12_Session.vi. Chaque exercice ajoute une commande de la face avant et un cas d'événement. A la fin du niveau, le programme règle entièrement un signal standard.

Exercice 13 — Un interrupteur « Sortie »

Ce qu'on apprend : utiliser un interrupteur (deux états stables) et la nouvelle valeur fournie par l'événement.

Enoncé. Enregistrez une copie de l'exercice 12 sous le nom Ex13_Sortie.vi.

1. Ajoutez un interrupteur « Sortie » (il garde sa position quand on le lâche).
2. Interrupteur en haut : envoyer `:OUTP ON`. Interrupteur en bas : envoyer `:OUTP OFF`.
3. Après chaque changement, relire l'état réel avec `:OUTP?` et l'afficher sur la LED « Sortie active ».
4. Après un clic sur Reset, l'interrupteur doit revenir en position basse.
    - ❖ Q1. Quelle action mécanique choisissez-vous pour l'interrupteur ? En quoi diffère-t-elle de celle des boutons précédents ?
    - ❖ Q2. Pourquoi relire `:OUTP?` au lieu de se fier à la position de l'interrupteur ?

Corrigé.

1. Face avant : un interrupteur à bascule (palette Booléen), action mécanique **Switch When Pressed** (commutation à l'appui). Contrairement aux boutons précédents, il garde sa position.
2. Nouveau cas « Sortie : Valeur changée ». A gauche du cas, le nœud de données d'événement fournit **NewVal**, la nouvelle position de l'interrupteur.
3. *Sélectionner* : si NewVal est Vrai, la commande est `:OUTP ON`, sinon `:OUTP OFF` → Interroger.vi.
4. Interroger(`:OUTP?`) → *Chaîne décimale en nombre* → *Egal à 1 ?* → LED Sortie active.
5. Dans le cas Reset de l'exercice 10, ajouter une variable locale de l'interrupteur (clic droit › Créer › Variable locale) mise à Faux : après `*RST`, l'interrupteur revient en position basse, comme la sortie.

L'interrupteur et la LED bougent ensemble, et le voyant de sortie du générateur suit.

Pourquoi relire avec `:OUTP?` ? L'interrupteur indique ce qu'on a **demandé** ; la LED indique ce que l'appareil a **fait**. Si la commande est refusée, les deux ne coïncident plus et on le voit tout de suite.

Exercice 14 — Régler la fréquence

Ce qu'on apprend : construire une commande à partir d'un nombre, avec le bon séparateur décimal.

Enoncé. Enregistrez une copie sous le nom Ex14_Frequence.vi.

1. Ajoutez une commande numérique « Fréquence (Hz) », un bouton « Appliquer » et un indicateur « Fréquence lue (Hz) ».
2. Au clic sur Appliquer, le VI construit la commande `:FREQ` avec la valeur saisie, l'envoie, puis relit la fréquence avec `:FREQ?` et l'affiche dans Fréquence lue.
3. Testez avec 1 000 Hz, 2 500,5 Hz et 50 000 Hz.
   - ❖ Q1. Pour chaque essai, recopiez la commande envoyée et la fréquence lue.
   - ❖ Q2. Quelle chaîne de format utilisez-vous dans *Formater en chaîne* ? Que se passe-t-il sans `%.;` sur un PC réglé en français ?

Corrigé.

1. Face avant : commande « Fréquence (Hz) » en DBL, plage 0,00001 à 31 000 000 (clic droit › Propriétés › Plage des données) ; bouton « Appliquer » (Latch) ; indicateur « Fréquence lue (Hz) ».
2. Cas « Appliquer : Valeur changée » : *Formater en chaîne* avec la chaîne de format `%.;:FREQ %.3f` et la commande Fréquence → Interroger.vi.
3. Interroger(`:FREQ?`) → *Chaîne fractionnaire en nombre* → Fréquence lue.

Consigne	Commande envoyée	Fréquence lue
1 000 Hz	`:FREQ 1000.000`	1 000
2 500,5 Hz	`:FREQ 2500.500`	2 500,5
50 kHz	`:FREQ 50000.000`	50 000

Erreur fréquente : oublier `%.;` au début de la chaîne de format. Sur un Windows en langue française, LabVIEW écrit alors `:FREQ 2500,500`. L'appareil ne reconnaît pas la virgule : la commande est refusée ou mal comprise. On le voit en comparant Fréquence lue à la consigne.

Exercice 15 — Amplitude et offset, avec contrôle des limites

Ce qu'on apprend : refuser une consigne interdite **avant** de l'envoyer, grâce à un sous-VI de contrôle.

Enoncé. Enregistrez une copie sous le nom Ex15_Niveaux.vi.

1. Ajoutez deux commandes : « Amplitude (Vcc) » et « Offset (V) », ainsi qu'une LED rouge « Niveaux interdits ».
2. Créez un sous-VI Vérifier_niveaux.vi qui calcule Vcc/2 + |offset| et indique si le résultat est inférieur ou égal à 5 V.
3. Au clic sur Appliquer : si le réglage est autorisé, envoyer fréquence, amplitude et offset ; sinon, ne rien envoyer et allumer la LED rouge.
4. Testez les quatre couples (amplitude ; offset) suivants : (2 ; 0), (6 ; 2), (8 ; 1,5), (10 ; 0).
    - ❖ Q1. Pour chaque couple, calculez Vcc/2 + |offset| et indiquez si le réglage est envoyé ou refusé.
    - ❖ Q2. Pourquoi est-il préférable de refuser le réglage avant de l'envoyer, plutôt que d'attendre l'erreur de l'appareil ?

Corrigé.

1. Sous-VI Vérifier_niveaux.vi : entrées Amplitude et Offset ; sortie booléenne « Autorisé ».
    - ❖ *Diviser* Amplitude par 2, *Valeur absolue* de Offset, *Additionner* les deux.
    - ❖ *Inférieur ou égal ?* à 5 → Autorisé.
2. Dans le cas Appliquer : Vérifier_niveaux.vi, puis une structure Condition sur Autorisé.
    - ❖ Cas **Vrai** : *Formater en chaîne* `%.;:FREQ %.3f;:VOLT:AMPL %.3f;:VOLT:OFFS %.3f` → Interroger.vi ; LED Niveaux interdits éteinte.
    - ❖ Cas **Faux** : rien n'est envoyé ; LED Niveaux interdits allumée.

Amplitude (Vcc)	Offset (V)	Vcc/2 + abs(offset)	Envoyé ?
2	0	1	oui
6	2	5	oui, juste à la limite
8	1,5	5,5	non, LED rouge
10	0	5	oui

En complément : le manuel impose une limite plus stricte pour les petites amplitudes : 0,5 V entre 0,1 et 0,999 Vcc, et 0,05 V en dessous de 0,1 Vcc. Ajouter une structure Condition dans le sous-VI pour choisir la bonne limite. Avec 0,5 Vcc et 0,3 V d'offset, on trouve 0,55 V > 0,5 V : refusé.

Exercice 16 — Choisir la forme d'onde

Ce qu'on apprend : un menu déroulant énuméré et une limite qui dépend du choix de l'utilisateur.

Enoncé. Enregistrez une copie sous le nom Ex16_Forme.vi.

1. Ajoutez un menu déroulant « Forme » avec trois choix : Sinus, Carré, Triangle.
2. Au clic sur Appliquer, la forme est envoyée avec les autres réglages (commande `:FUNC`).
3. Pour le triangle, la fréquence ne doit pas dépasser 500 kHz. Si elle dépasse, rien n'est envoyé et la LED rouge (renommée « Réglage interdit ») s'allume.
4. Testez un triangle à 100 kHz, puis un triangle à 1 MHz.
    - ❖ Q1. Quel mot SCPI correspond à chaque forme ? Quelle est la fréquence maximale de chacune ?
    - ❖ Q2. Qu'observez-vous pour chacun des deux essais ?
    - ❖ Q3. Pourquoi envoie-t-on `:FUNC` avant `:FREQ` dans le même message ?

Corrigé.

1. Face avant : *Menu déroulant énuméré* (palette Anneau et énumération). Clic droit › *Editer les éléments* : Sinus (0), Carré (1), Triangle (2).
2. Dans le cas Appliquer, une structure Condition reliée à Forme fournit deux valeurs par cas :

Cas	Mot SCPI	Fréquence maximale
Sinus	SIN	31 000 000 Hz
Carré	SQU	31 000 000 Hz
Triangle	TRI	500 000 Hz

3. Ajouter un test au contrôle de l'exercice 15 : Fréquence ≤ fréquence maximale. L'envoi n'a lieu que si les deux conditions sont vraies (*Et*). Renommer la LED « Réglage interdit ».
4. La commande devient : *Formater en chaîne* avec `%.;:FUNC %s;:FREQ %.3f;:VOLT:AMPL %.3f;:VOLT:OFFS %.3f`. La première entrée est le mot SCPI.

Le triangle à 100 kHz est envoyé ; à 1 MHz, la LED rouge s'allume et rien ne part.

Pourquoi envoyer la forme avant la fréquence ? Le manuel précise qu'un changement de forme peut rendre la fréquence courante hors limites. Envoyer `:FUNC` puis `:FREQ` dans le même message évite cet état intermédiaire.

Exercice 17 — Un journal des erreurs

Ce qu'on apprend : lire la file d'erreurs de l'appareil après chaque action et garder un historique.

Enoncé. Enregistrez une copie sous le nom Ex17_Journal.vi.

1. Créez un sous-VI Lire_erreurs.vi : il envoie `:SYST:ERR?` en boucle jusqu'à recevoir le code 0, et renvoie la liste des messages d'erreur reçus.
2. Appelez Lire_erreurs.vi à la fin de chaque cas d'événement.
3. Affichez chaque message, précédé de l'heure, dans un indicateur chaîne « Journal ».
4. Pour tester, ajoutez une console : un champ « Commande libre » et un bouton « Envoyer ». Envoyez `:FREQUENCE 1000`.
    - ❖ Q1. Quelle ligne apparaît dans le Journal ? Que signifie le code ?
    - ❖ Q2. Une commande correcte ajoute-t-elle une ligne au Journal ?

- ❖ Q3. Pourquoi limiter la boucle de Lire_erreurs.vi à 20 tours ?

Corrigé.

Lire_erreurs.vi :

1. Une boucle While. A chaque tour : Interroger(`:SYST:ERR?`).
2. *Balayer une chaîne* avec le format `%d,%s` : le code (entier) et le message.
3. Condition d'arrêt : code = 0.
4. Si le code est différent de 0, le message est ajouté à un tableau de chaînes (tunnel auto-indexé conditionnel, ou *Construire un tableau* dans un registre à décalage).
5. Sécurité : arrêter aussi la boucle après 20 tours, au cas où la réponse serait illisible.

Programme principal : à la fin de chaque cas, Lire_erreurs.vi. Pour chaque message, *Formater date/heure en chaîne* (`%H:%M:%S`) + message + saut de ligne, ajouté au texte déjà présent dans le Journal. Le texte est conservé dans un registre à décalage de la boucle principale.

Une ligne du type `14:05:12 -113,"Undefined header"` apparaît après `:FREQUENCE 1000`. Le numéro et le libellé exacts sont ceux de l'appareil : la norme SCPI réserve les codes −100 à −199 aux erreurs de commande. Les commandes correctes n'ajoutent rien au journal.

Exercice 18 — Mémoriser et rappeler des configurations

Ce qu'on apprend : utiliser les 49 mémoires de l'appareil et choisir la configuration de démarrage.

Enoncé. Enregistrez une copie sous le nom Ex18_Memoires.vi.

1. Ajoutez une commande « Numéro de mémoire » (1 à 49) et trois boutons : « Enregistrer », « Rappeler » et « Réglage usine ».
2. Enregistrez deux configurations : mémoire 1 = sinus 1 kHz, 2 Vcc ; mémoire 2 = carré 10 kHz, 4 Vcc.
3. Rappelez la mémoire 1, puis la mémoire 2, et observez le générateur et la face avant.
4. Faites démarrer le générateur sur la mémoire 1, puis éteignez-le et rallumez-le.
   - ❖ Q1. Quelle commande envoie chaque bouton ?
   - ❖ Q2. Après un rappel, que faut-il faire pour que la face avant affiche les bons réglages ?
   - ❖ Q3. Quelle commande choisit la configuration au démarrage ? Sur quel signal le générateur redémarre-t-il ?

Corrigé.

Bouton	Commandes	Ensuite
Enregistrer	`*SAV <n>`	Lire_erreurs.vi
Rappeler	`*RCL <n>`	Lire_reglages.vi pour mettre la face avant à jour (sous-VI qui relit :FUNC?, :FREQ?, :VOLT:AMPL?, :VOLT:OFFS? et :OUTP?)
Réglage usine	`*RCL 0`	Lire_reglages.vi
Démarrage sur la mémoire 1	`:SYST:POB 1`	—

Bouton	Commandes	Ensuite

1. Contrôle : n entre 1 et 49 pour Enregistrer (la mémoire 0 est le réglage usine, en lecture seule).
2. **Point clé :** après un rappel, l'appareil a changé de réglage sans que la face avant le sache. Il faut appeler Lire_reglages.vi, sinon l'écran affiche des valeurs fausses.
3. `:SYST:POB` choisit la configuration au démarrage : 0 = usine, 1 à 49 = une mémoire, 50 = dernier réglage avec sortie coupée, 51 = dernier réglage avec l'état de sortie.

Rappeler 1 et Rappeler 2 font basculer l'appareil et la face avant entre sinus et carré. Après une mise hors tension puis sous tension, le générateur démarre sur le sinus de 1 kHz.

Bilan du niveau 3 : le programme règle la sortie, la fréquence, l'amplitude, l'offset et la forme ; il refuse les réglages interdits, trace les erreurs et mémorise des configurations. Le niveau 4 le relie à l'oscilloscope.

Niveau 4 — Vérifier le signal avec l'oscilloscope

Exercice 19 — Vérifier le signal avec l'oscilloscope

Ce qu'on apprend : piloter un second appareil sur le même bus et confronter la consigne à la mesure.

Enoncé. Enregistrez une copie sous le nom Ex19_Verifier.vi.

1. Avant la boucle, ouvrez une seconde session VISA vers l'oscilloscope 5062DC (GPIB0::7::INSTR).
2. Après chaque clic sur Appliquer, le programme règle les calibres de l'oscilloscope, puis lit la fréquence et l'amplitude crête à crête de CH1.
3. Affichez les deux mesures, leur écart à la consigne en %, et une LED verte « Conforme » allumée si les deux écarts sont inférieurs à 3 %.
4. Faites trois essais : 1 kHz, 2 Vcc ; 100 kHz, 1 Vcc ; puis 1 kHz, 2 Vcc sans le bouchon 50 Ω.
    - ❖ Q1. Pour chaque essai, notez la fréquence mesurée, la Vcc mesurée et l'état de la LED Conforme.
    - ❖ Q2. Expliquez le résultat du troisième essai.
    - ❖ Q3. Que répond l'oscilloscope à *IDN? ? A quelle famille appartient-il : GDS-2000 ou GDS-2000A ?

Corrigé.

1. Avant la boucle : ouvrir une **seconde** session VISA sur l'oscilloscope, qui entre dans la boucle par un troisième registre à décalage. Interroger.vi fonctionne tel quel avec cette session : c'est l'intérêt du sous-VI.
2. Dans le cas Appliquer, après l'envoi au générateur :

```
Oscilloscope :CHANnel1:COUPling 1;:CHANnel1:SCALe <V/div>;:TIMebase:SCALe
<s/div>
(attente 1 000 ms)
```

```
Oscilloscope :MEASure:SOURce 1;:MEASure:FREQuency?   → f mesurée
Oscilloscope :MEASure:SOURce 1;:MEASure:VPP?         → Vcc mesurée
```

3. Calibres : V/div ≈ (Vcc/2 + abs(offset)) / 3, arrondi à la valeur 1-2-5 supérieure ; s/div = 0,2 / f.
4. Ecarts : 100 × (mesure − consigne) / consigne, puis *Valeur absolue* et *Inférieur ?* 3 ; *Et* des deux résultats → LED Conforme.
5. Après la boucle : fermer aussi la session de l'oscilloscope.

Consigne	f mesurée	Vcc mesurée	Conforme
1 kHz, 2 Vcc	≈ 1 000 Hz	≈ 2 V	oui
100 kHz, 1 Vcc	≈ 100 kHz	≈ 1 V	oui
1 kHz, 2 Vcc, **sans bouchon 50 Ω**	≈ 1 000 Hz	≈ 4 V	**non** : l'amplitude double sur 1 MΩ

A vérifier au premier essai : les commandes de l'oscilloscope suivent la famille GDS-2000 (voir le banc de mesure et l'annexe). Si `*IDN?` répond « GDS-2000A », utiliser les commandes du manuel GDS-2000A (par exemple `:MEASure:SOURce1 CH1;:MEASure:PK2Pk?`).

TP9 — Mise en communication GPIB

Objectif : vérifier que le PC dialogue avec les deux appareils, puis écrire le sous-VI de dialogue utilisé dans tous les TP suivants.

Travail demandé

1. **Régler les adresses.** Sur l'oscilloscope, réglez l'interface sur GPIB et l'adresse sur 7. Sur le générateur, vérifiez que l'adresse est 9.
2. **Trouver les appareils.** Dans NI MAX, ouvrez Périphériques et interfaces › GPIB0, puis cliquez sur « Rechercher les instruments ».
   - ❖ Q1. Recopiez les deux noms de ressources trouvés.
3. **Identifier chaque appareil.** Pour chacun, ouvrez « Communiquer avec l'instrument » et envoyez `*IDN?`.
   - ❖ Q2. Recopiez les deux réponses. D'après la réponse de l'oscilloscope, à quelle famille appartient-il : GDS-2000 ou GDS-2000A ?
4. **Premiers tests.** Envoyez au générateur `*RST`, puis `*TST?`, puis `:OUTP ON`. Sur l'oscilloscope, appuyez sur la touche AUTOSET, puis envoyez `:MEASure:SOURce 1;:MEASure:FREQuency?`.
   - ❖ Q3. Que répond le générateur à `*TST?` ? Que signifie cette réponse ?
   - ❖ Q4. Quelle fréquence mesure l'oscilloscope ? D'où vient ce signal, alors qu'on n'a réglé aucune fréquence ?
5. **Sous-VI de dialogue.** Créez Interroger.vi avec deux entrées (« Ressource », « Commande »), une sortie (« Réponse »), ainsi qu'error in et error out. Il envoie la commande, et ne lit une réponse que si la commande contient un « ? ».
   - ❖ Q5. Pourquoi ne faut-il pas lire de réponse après une commande sans « ? », comme `*RST` ?
6. **Identifier les deux appareils d'un coup.** Créez Identifier.vi, qui appelle Interroger.vi deux fois (une fois par appareil) avec `*IDN?` et affiche les deux réponses.

Corrigé

- **NI MAX** doit lister `GPIB0::7::INSTR` et `GPIB0::9::INSTR`. Si un appareil manque, vérifier son adresse, le câble et que son interface active est bien GPIB.

Commande	Appareil	Réponse attendue
`*IDN?`	générateur	`SEFRAM,SEFRAM 4463,0,V1.1` (la version peut varier)
`*IDN?`	oscilloscope	fabricant, modèle, numéro de série, version : c'est ici qu'on lit s'il s'agit d'un GDS-2000 ou d'un GDS-2000A
`*TST?`	générateur	`0` (autotest réussi)
`:MEASure:FREQuency?`	oscilloscope	environ 1 MHz : après `*RST`, le générateur produit un sinus de 1 MHz et 5 Vcc

- **Interroger.vi :** *VISA Open* (Ressource) → *Concaténer des chaînes* (Commande + constante *Saut de ligne*) → *VISA Write*. Puis une structure Condition sur *Rechercher une sous-chaîne* (« ? » trouvé ou non) : dans le cas Vrai, *VISA Read* de 1 000 octets → Réponse ; dans le cas Faux, chaîne vide. Enfin *VISA Close*. Le fil d'erreur traverse tout. Connecteur : Ressource, Commande et error in à gauche ; Réponse et error out à droite.
- **Identifier.vi :** deux appels à Interroger.vi, un par ressource, avec la commande `*IDN?`. Le fil d'erreur du premier va vers le second, puis vers le *Gestionnaire d'erreur simple*.
- **Remarque :** cette version d'Interroger.vi ouvre et ferme la session à chaque appel (entrée « Ressource »). Celle de l'exercice 12 reçoit au contraire une session déjà ouverte (entrée « référence VISA »). Si les deux servent dans le même projet, donnez-leur des noms différents.
- **Si `*IDN?` ne répond pas (timeout) :** mauvaise adresse, appareil réglé en RS-232 ou USB, ou commande envoyée sans saut de ligne final.

TP10 — Piloter le générateur et mesurer à l'oscilloscope

Objectif : régler un signal sur le SEFRAM 4463 depuis LabVIEW, le faire mesurer par le 5062DC et comparer consigne et mesure.

Travail demandé

1. **Face avant.** Créez Sinus_Mesure.vi avec : trois commandes (Fréquence en Hz, Amplitude en Vcc, Offset en V), un bouton « Appliquer », et quatre indicateurs (Fréquence mesurée, Vcc mesurée, Ecart en %, Erreur générateur).
2. **Ce que fait le bouton Appliquer**, dans cet ordre :
   - ❖ envoyer le sinus au générateur ;
   - ❖ adapter le calibre vertical et la base de temps de l'oscilloscope ;
   - ❖ attendre 1 seconde ;
   - ❖ lire la fréquence et l'amplitude mesurées par l'oscilloscope ;
   - ❖ lire la file d'erreurs du générateur.
3. **Essais.** Avant de lancer chaque essai du tableau ci-dessous, calculez Vcc/2 + |offset| et prévoyez si le générateur va accepter le réglage (la limite est 5 V).
   - ❖ Q1. Pour chaque essai, notez : accepté ou refusé, fréquence mesurée, Vcc mesurée, écart en %.

❖ Q2. Pour l'essai refusé, quelle est la réponse de `:SYST:ERR?` ? Quel réglage reste en place sur le générateur ?

Essai	Fréquence	Amplitude	Offset
a	1 kHz	2 Vcc	0 V
b	10 kHz	4 Vcc	0 V
c	100 kHz	1 Vcc	0 V
d	1 kHz	6 Vcc	2 V
e	1 kHz	8 Vcc	1,5 V

4. **Effet du bouchon 50 Ω.**
   ❖ Q3. Refaites l'essai a après avoir retiré le bouchon 50 Ω. Quelle amplitude mesure l'oscilloscope ? Expliquez en deux phrases pourquoi elle change.

Corrigé

- **Architecture :** une boucle While avec une structure Evénement (cas « Appliquer : Valeur changée » et cas STOP). Le cas « Appliquer » appelle Interroger.vi (TP9) plusieurs fois, dans l'ordre imposé par le fil d'erreur.
- **Commandes envoyées à chaque clic :**

```
Générateur   :FUNC SIN;:FREQ <f>;:VOLT:AMPL <A>;:VOLT:OFFS <o>;:OUTP ON
Générateur   :SYST:ERR?
Oscilloscope :CHANnel1:COUPling 1;:CHANnel1:SCALe <V/div>;:TIMebase:SCALe
<s/div>
(attente 1 000 ms)
Oscilloscope :MEASure:SOURce 1;:MEASure:FREQuency?
Oscilloscope :MEASure:SOURce 1;:MEASure:VPP?
```

- **Calibres :** V/div ≈ (A/2 + |offset|) / 3, arrondi à la valeur 1-2-5 supérieure (0,5 ; 1 ; 2 V/div) : le signal reste dans l'écran. s/div ≈ 0,2 / f : quelques périodes à l'écran.
- **Ecart :** Ecart (%) = 100 × (Vcc mesurée − A) / A. *Chaîne fractionnaire en nombre* convertit chaque réponse en nombre.

Essai	Accepté ?	Fréquence mesurée	Vcc mesurée	Remarque
a	oui	≈ 1 000 Hz	≈ 2 V	Vcc/2 + offset = 1 ≤ 5
b	oui	≈ 10 kHz	≈ 4 V	2 ≤ 5
c	oui	≈ 100 kHz	≈ 1 V	0,5 ≤ 5
d	oui	≈ 1 000 Hz	≈ 6 V	3 + 2 = 5 ≤ 5 : signal entre −1 V et +5 V
e	**non**	inchangée	inchangée	4 + 1,5 = 5,5 > 5 : `:SYST:ERR?` renvoie un code d'erreur négatif et l'ancien réglage reste en place

Les écarts doivent rester de quelques pour cent : précision du générateur (± 1 % ± 20 mV) et de l'oscilloscope. Dans l'essai d, le couplage doit être DC pour voir l'offset.

- **Sans bouchon :** l'oscilloscope mesure environ **4 V** au lieu de 2. Le générateur a une résistance interne de 50 Ω ; sur 50 Ω, la moitié de sa tension interne se retrouve aux bornes de la charge. Sur 1 MΩ, on récupère presque toute la tension interne, soit le double.

TP11 — Rapatrier le signal de l'oscilloscope sur le PC

Objectif : envoyer un sinus au générateur, puis récupérer les points capturés par l'oscilloscope et les tracer sur un graphe LabVIEW.

Montage : celui du banc, avec le bouchon 50 Ω sur CH1.

Travail demandé

1. **Face avant.** Créez Rapatrier.vi avec deux commandes, Fréquence (1 000 Hz) et Amplitude (2 Vcc), un graphe « Signal récupéré » et trois indicateurs : dt (temps entre deux points), Nombre de points et Veff (valeur efficace).
2. **Produire et capturer le signal.** Le VI envoie le sinus au générateur, règle l'oscilloscope, attend 1 seconde, puis fige l'oscilloscope (touche STOP par programme).
3. **Rapatrier les points.** Le VI demande à l'oscilloscope les points de la voie CH1 avec `:ACQuire1:MEMory?`, les convertit en volts et trace le signal.
   - ❖ Q1. Combien de points recevez-vous ? Quelle valeur de dt lisez-vous ? Quelle durée de signal le graphe affiche-t-il ?
   - ❖ Q2. Le signal tracé a-t-il la bonne période (1 ms) et la bonne amplitude (2 V crête à crête) ?
4. **Calculer la valeur efficace.**
   - ❖ Q3. Calculez Veff avec LabVIEW et comparez à la valeur théorique Vcc / (2√2). Donnez l'écart en %.
5. **Enregistrer.**
   - ❖ Q4. Enregistrez le signal (temps, tension) dans un fichier tableur, puis ouvrez-le dans Excel pour retracer la courbe.

Corrigé : les blocs dans l'ordre (tous reliés par le fil d'erreur, sans boucle)

1. **Générateur :** Interroger.vi avec `:FUNC SIN;:FREQ <f>;:VOLT:AMPL <A>;:VOLT:OFFS 0;:OUTP ON`.
2. **Oscilloscope :** `:CHANnel1:COUPling 1;:CHANnel1:SCALe <A/4>;:CHANnel1:OFFSet 0;:TIMebase:SCALe <0,2/f>;:TRIGger:SOURce 0;:TRIGger:MODe 1;:ACQuire:LENGth 0;:RUN`, puis *Attendre (ms)* 1 000 et `:STOP`.
3. **Vcc de référence :** `:MEASure:SOURce 1;:MEASure:VPP?` → Vcc mesurée (sert à la conversion en volts).
4. **Demande des points :** *VISA Open* (oscilloscope), *VISA Write* `:ACQuire1:MEMory?\n`.
5. **Lecture binaire :** nœud de propriété VISA *Termination Character Enable* = Faux, car les données peuvent contenir l'octet 0x0A. *VISA Read* 2 octets (`#` et un chiffre d) ; *VISA Read* d octets → taille S (`1008` pour 500 points) ; *VISA Read* S octets → bloc ; *VISA Flush I/O Buffer* ; *VISA Close*.
6. **Décodage du bloc :**

Octets	Contenu	Bloc LabVIEW
0 à 3	dt, réel 32 bits, octet de poids faible en premier	*Sous-ensemble d'une chaîne* (0, 4) → *Redresser une chaîne*, type SGL, *little-endian*
4	numéro de voie	non utilisé
5 à 7	réservés	non utilisés
8 à fin	points : entiers signés 16 bits, octet de poids fort en premier	*Sous-ensemble d'une chaîne* (8, S − 8) → *Redresser une chaîne*, tableau de I16, *big-endian*, « la chaîne contient la taille » = Faux

7. **Conversion en volts :** k = Vcc mesurée / (max − min des points), avec *Max & Min d'un tableau* ; volts = k × points. (Valeur de contrôle souvent rencontrée sur cette famille d'oscilloscopes : 25 points par division, soit k = 0,5 / 25 = 0,02 V par point à 0,5 V/div.)
8. **Tracé :** *Construire une waveform* (t0 = 0, dt, Y = volts) → graphe.
9. **Valeur efficace :** *RMS.vi* (palette Probabilités et statistiques) sur le tableau de volts.
10. **Fichier :** *Ecrire dans un fichier tableur* avec deux colonnes, temps (i × dt) et tension.
11. **Fin :** `:RUN` à l'oscilloscope, *Gestionnaire d'erreur simple*.

Annexe — Aide-mémoire

Raccourcis LabVIEW

Raccourci	Action	Raccourci	Action
Ctrl + E	face avant ↔ diagramme	Ctrl + R	exécuter
Ctrl + H	aide contextuelle	Ctrl + .	arrêter
Ctrl + B	supprimer les fils brisés	Ctrl + U	nettoyer le diagramme
Ctrl + Espace	Quick Drop (fonction par son nom)	Ctrl + T	fenêtres côte à côte

Générateur SEFRAM 4463 (commandes utilisées)

Commande	Effet
`*IDN?`	identification
`*RST`	réglages usine : sinus 1 MHz, 5 Vcc, sortie coupée
`*TST?`	autotest, 0 = réussi
`:FUNC SIN` (ou `SQU`, `TRI`)	forme d'onde
`:FREQ <f>`	fréquence en Hz (sinus jusqu'à 31 MHz)

Commande	Effet
`:VOLT:AMPL <Vcc>`	amplitude, 10 mV à 10 Vcc sur 50 Ω
`:VOLT:OFFS <V>`	offset, avec Vcc/2 + abs(offset) ≤ 5 V
`:OUTP ON` / `:OUTP OFF`	sortie active ou coupée
`:SYST:ERR?`	lit la file d'erreurs, `0,"No error"` si vide

Oscilloscope SEFRAM 5062DC

Commandes de la famille GDS-2000. Vérifiez d'abord la réponse à *IDN? : si l'oscilloscope répond « GDS-2000A », utilisez le manuel de programmation GDS-2000A.

Rôle	GDS-2000 (caractéristiques du 5062DC)
Couplage continu	`:CHANnel1:COUPling 1`
Calibre vertical	`:CHANnel1:SCALe 0.5`
Position verticale	`:CHANnel1:OFFSet 0`
Base de temps	`:TIMebase:SCALe 2e-4`
Déclenchement sur CH1	`:TRIGger:SOURce 0`
Mode auto	`:TRIGger:MODe 1`
Lancer / figer	`:RUN` / `:STOP`
Fréquence mesurée	`:MEASure:SOURce 1;:MEASure:FREQuency?`
Vcc mesurée	`:MEASure:SOURce 1;:MEASure:VPP?`
Longueur d'enregistrement	`:ACQuire:LENGth 0` (500 points)
Points de CH1	`:ACQuire1:MEMory?` : en-tête binaire de 8 octets (dt, voie), puis les points

Sources :

Manuel d'utilisation et de programmation SEFRAM 4463 ;

[Manuel de programmation GW Instek GDS-2000]